11.1

一、解题过程

1. 问题背景与数据特点

本题考察原油辛烷值 y 与七种成分含量 $x_1, x_2, \cdots, x_7$ 的关系，数据满足成分约束 $x_1 + x_2 + \cdots + x_7 = 1$。样本容量仅为 12，自变量个数较多且可能存在多重共线性，普通最小二乘法难以应用。偏最小二乘（Partial Least Squares, PLS）回归能有效处理自变量高度相关及小样本问题，因此选用 PLS 方法建立回归方程。

2. 偏最小二乘回归原理

PLS 回归同时提取自变量 X 和因变量 y 的主成分，要求所提取的成分之间的协方差最大。设提取 h 个成分 $t_1, t_2, \cdots, t_h$，建立 y 对 t 的回归，再还原为原始变量的回归方程。模型内部对 X 和 y 进行标准化处理，最后需将系数还原至原始量纲。

3. 确定最佳主成分个数

采用留一交叉验证（Leave-One-Out Cross Validation, LOOCV）。对可能的主成分个数 $n = 1,2,\cdots,5$，依次计算交叉验证的决定系数 R^2_{CV}，选择 R^2_{CV} 最大的 n 作为最佳成分个数。

4. 建立最终模型并还原原始系数

以最佳主成分个数建立 PLS 模型，拟合全部数据。使用 StandardScaler 手动对 X 和 y 标准化，拟合后得到标准化回归系数。再根据自变量的均值 $\bar{x}_j$、标准差 s_{x_j} 以及因变量的均值 $\bar{y}$ 和标准差 s_y，将系数还原为原始变量下的回归方程：

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_7 x_7$$

还原公式为：

$$\beta_j = s_y \cdot \frac{\beta_j^{\text{std}}}{s_{x_j}}, \beta_0 = \bar{y} - \sum_{j=1}^{7} \beta_j \bar{x}_j$$

其中 β_j^{std} 为标准化系数。

5. 模型评价

使用训练集拟合决定系数 R^2 和交叉验证决定系数 R^2_{CV} 评估模型预测能力和解释能力。

二、结果呈现

1. 交叉验证结果

主成分个数	交叉验证 R^2_{cv}
1	0.8809
2	0.9114
3	**0.9567**
4	0.9546
5	0.9476

最佳主成分个数为 **3**。

2. 最终回归方程

使用 3 个主成分拟合全部数据，模型训练集决定系数 $R^2 = 0.9906$，说明模型对训练数据拟合效果极佳，且交叉验证 $R^2 = 0.9567$，表明模型具有良好的预测能力。

原始量纲下的回归方程为：

$$\hat{y} = 92.6760 - 9.8283x_1 - 6.9602x_2 - 16.6662x_3 - 8.4218x_4 - 4.3889x_5 + 10.1613x_6 - 34.5290x_7$$

3. 系数解释（相对影响）

- x_1（直接蒸馏成分）：系数为负，其他条件相对变化时，该成分比例增加会降低辛烷值。
- x_2（重整汽油）：系数为负，对辛烷值有负面影响。
- x_3（原油热裂化油）：系数为负且绝对值较大，是降低辛烷值的主要因素之一。
- x_4（原油催化裂化油）：系数为负，影响程度中等。
- x_5（聚合物）：系数为负，但绝对值较小。
- x_6（烷基化物）：系数为正，是唯一对辛烷值有正向贡献的成分，增加其比例有助于提高辛烷值。
- x_7（天然香精）：系数为负且绝对值最大，对辛烷值的负向影响最为显著。

注：由于自变量满足 $\sum x_i = 1$，各成分比例之间相互制约，单个系数不能解释为“其他变量不变”时的边际效应，但系数的大小和符号仍能反映各成分对辛烷值的相对影响方向与强度。

三、代码展示

```
import numpy as np
import pandas as pd
from sklearn.cross_decomposition import PLSRegression
```

```
from sklearn.model_selection import LeaveOneOut, cross_val_predict
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler

data = {
    'x1': [0, 0, 0, 0, 0, 0, 0.17, 0.17, 0.17, 0.17, 0.21, 0],
    'x2': [0.23, 0.1, 0, 0.49, 0, 0.62, 0.27, 0.19, 0.21, 0.15, 0.36, 0],
    'x3': [0, 0, 0, 0, 0, 0, 0.1, 0.1, 0.1, 0.1, 0.12, 0],
    'x4': [0, 0, 0.1, 0, 0.62, 0, 0.38, 0.38, 0.38, 0.38, 0.25, 0.55],
    'x5': [0, 0.12, 0.12, 0.12, 0.12, 0, 0, 0.02, 0, 0.02, 0, 0],
     'x6': [0.74, 0.74, 0.74, 0.37, 0.18, 0.37, 0, 0.06, 0.06, 0.1, 0,
0.37],
    'x7': [0.03, 0.04, 0.04, 0.02, 0.08, 0.01, 0.08, 0.08, 0.08, 0.08,
0.06, 0.08],
    'y':  [98.7, 97.8, 96.6, 92.0, 86.6, 91.2, 81.9, 83.1, 82.4, 83.2,
81.4, 88.1]
}
df = pd.DataFrame(data)
X = df[['x1','x2','x3','x4','x5','x6','x7']].values
y = df['y'].values.reshape(-1,1)

loo = LeaveOneOut()
print("=== 交叉验证结果 ===")
best_n = 1
best_r2 = -np.inf
max_components = min(5, X.shape[0]-1)   # 最多 5 个成分，且不超过样本数-1

for n in range(1, max_components+1):
    pls = PLSRegression(n_components=n)
    y_pred = cross_val_predict(pls, X, y, cv=loo)
    r2 = r2_score(y, y_pred)
    print(f"成分个数 = {n}, 交叉验证 R² = {r2:.4f}")
    if r2 > best_r2:
        best_r2 = r2
        best_n = n

print(f"\n最佳主成分个数 = {best_n}\n")

scaler_x = StandardScaler()
scaler_y = StandardScaler()
X_std = scaler_x.fit_transform(X)
y_std = scaler_y.fit_transform(y).ravel()

pls_final = PLSRegression(n_components=best_n)
```

```
pls_final.fit(X_std, y_std)

coef_std = pls_final.coef_.ravel()

x_mean = scaler_x.mean_
x_scale = scaler_x.scale_
y_mean = scaler_y.mean_[0]
y_scale = scaler_y.scale_[0]

raw_coef = y_scale * coef_std / x_scale
raw_intercept = y_mean - np.sum(raw_coef * x_mean)

print("=== 最终偏最小二乘回归方程 ===")
print(f"截距 β0 = {raw_intercept:.4f}")
for i, name in enumerate(['x1','x2','x3','x4','x5','x6','x7']):
    print(f"系数 β{i+1} ({name}) = {raw_coef[i]:.4f}")

# 计算训练集的拟合 R²（原始尺度下）
y_pred_std = pls_final.predict(X_std)
y_pred = scaler_y.inverse_transform(y_pred_std.reshape(-1,1)).ravel()
train_r2 = r2_score(y, y_pred)
print(f"\n 训练集拟合 R² = {train_r2:.4f}")

print(f"sklearn 直接提供的训练 R² = {pls_final.score(X_std, y_std):.4f}")
```

11.2

一、解题过程

1. 问题背景

题目中给出了 38 名学生的 7 个体质指标（x_1~x_7）和 5 项运动能力指标（y_1~y_5）。需要利用偏最小二乘（Partial Least Squares, PLS）回归建立运动能力与体质变量之间的定量关系模型，分析体质对运动能力的影响。

2. 偏最小二乘回归原理

PLS 回归能够同时处理多个因变量以及自变量之间的多重共线性。它通过提取自变量X和因变量Y的潜在成分，使得所提取的成分之间的协方差最大，然后建立Y对成分的回归，最后还原为原始变量的回归方程。由于各变量量纲不同，建模前需对X和Y进行标准化。

3. 确定最佳主成分个数

采用留一交叉验证（Leave-One-Out Cross Validation, LOOCV）。对主成分个数$h=$

$1,2,\cdots,6$，依次计算交叉验证的决定系数R^2_{CV}（对每个因变量分别计算后取平均值），选择平均R^2_{CV}最大的h作为最佳成分个数。

4. 建立最终模型并还原原始系数

以最佳主成分个数建立 PLS 模型，拟合全部数据。设标准化后的回归系数为β_j^{std}，自变量X_j的均值为$\bar{x}_j$、标准差为s_{x_j}，因变量Y_k的均值为$\bar{y}_k$、标准差为s_{y_k}，则原始尺度下的回归系数和截距为：

$$\beta_{kj} = s_{y_k} \cdot \frac{\beta_{kj}^{\text{std}}}{s_{x_j}}, \beta_{k0} = \bar{y}_k - \sum_{j=1}^{7} \beta_{kj}\, \bar{x}_j$$

最终得到每个运动能力y_k关于体质变量$x_1, \ldots, x_7$的线性回归方程。

5. 模型评价

使用训练集决定系数R^2和交叉验证平均R^2_{CV}评估模型的拟合优度与预测能力。

二、结果呈现

1. 交叉验证选择主成分个数

主成分个数	平均交叉验证 R^2_{cv}	各运动能力 R^2_{cv}（$y_1 \sim y_5$）
1	**0.2331**	0.1131, 0.3530, 0.3406, 0.1781, 0.1805
2	0.2098	0.1810, 0.3421, 0.2733, 0.1650, 0.0877
3	0.2252	0.2251, 0.3052, 0.2914, 0.1059, 0.1983
4	0.1787	0.1847, 0.2796, 0.2600, 0.0187, 0.1503
5	0.1496	0.1776, 0.2675, 0.2031, -0.0567, 0.1566
6	0.1457	0.1631, 0.2389, 0.1802, -0.0453, 0.1917

根据交叉验证结果，**最佳主成分个数为 1**（平均交叉验证 $R^2 = 0.2331$）。

2. 最终回归方程（原始量纲）

采用 1 个主成分建立 PLS 模型，模型对训练数据的拟合优度（各运动能力的 R^2）如下：

- y_1：$R^2 = 0.2445$
- y_2：$R^2 = 0.4581$
- y_3：$R^2 = 0.4282$

- y_4：$R^2 = 0.3192$
- y_5：$R^2 = 0.2812$

平均训练集 $R^2 = 0.3462$。

原始量纲下的回归方程为：

$$\hat{y}_1 = 9.5514 - 0.0178x_1 - 0.0107x_2 - 0.0046x_3 - 0.0048x_4 - 0.0004x_5 - 0.0066x_6 - 0.0012x_7$$

$$\hat{y}_2 = 35.7726 + 2.9636x_1 + 1.7776x_2 + 0.7618x_3 + 0.8041x_4 + 0.0748x_5 + 1.1086x_6 + 0.2054x_7$$

$$\hat{y}_3 = 2.8108 + 0.1834x_1 + 0.1100x_2 + 0.0471x_3 + 0.0498x_4 + 0.0046x_5 + 0.0686x_6 + 0.0127x_7$$

$$\hat{y}_4 = -22.1181 + 0.2178x_1 + 0.1307x_2 + 0.0560x_3 + 0.0591x_4 + 0.0055x_5 + 0.0815x_6 + 0.0151x_7$$

$$\hat{y}_5 = 596.6054 - 1.6860x_1 - 1.0113x_2 - 0.4334x_3 - 0.4575x_4 - 0.0426x_5 - 0.6307x_6 - 0.1169x_7$$

3. 结果分析

① 由于最佳主成分个数仅为 1，说明体质变量与运动能力之间的线性关系较弱，或存在较强的非线性特征。模型整体拟合优度偏低（平均训练 $R^2 = 0.3462$），预测能力（交叉验证 $R^2 = 0.2331$）也有限。

② 从各运动能力来看，y_2 和 y_3 的训练 R^2 相对较高（约 0.46、0.43），而 y_1、y_4、y_5 的拟合效果较差（$R^2 < 0.32$）。

③ 系数符号表明：对于 y_2、y_3、y_4，几乎所有体质指标均呈正向影响；而对于 y_1 和 y_5，体质指标多为负向影响。但鉴于模型拟合能力较低，这些系数的实际解释需谨慎。

三、代码展示

```
import numpy as np
import pandas as pd
from sklearn.cross_decomposition import PLSRegression
from sklearn.model_selection import LeaveOneOut, cross_val_predict
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler

np.random.seed(42)

data = {
    'x1':
[46,52,46,49,42,48,49,48,45,48,49,47,41,52,45,49,53,47,48,49,48,42,54,5
3,42,46,45,50,42,48,42,48,49,47,49,44,52,48],
    'x2':
[55,55,69,50,55,61,60,63,55,64,52,62,51,55,52,57,65,57,60,55,69,57,64,6
3,71,66,56,60,51,50,52,67,74,55,74,52,66,68],
```

```
    'x3':
[126,95,107,105,90,106,100,122,105,120,100,100,101,125,94,110,112,95,12
0,113,128,122,155,120,138,120,91,120,126,115,140,105,151,113,120,110,13
0,100],
    'x4':
[51,42,38,48,46,43,49,52,48,38,42,34,53,43,50,47,47,47,47,41,42,46,51,4
2,44,45,29,42,50,41,48,39,49,40,53,37,47,45],
    'x5':
[75.0,81.2,98.0,97.6,66.5,78.0,90.6,56.0,76.0,60.2,53.4,61.2,62.4,86.3,
51.4,72.3,90.4,72.3,86.4,84.1,47.9,54.2,71.4,56.6,65.2,62.2,66.2,56.6,5
0.0,52.9,56.3,69.2,54.2,71.4,54.5,54.9,45.9,53.6],
    'x6':
[25,18,18,16,2,25,15,17,15,20,6,10,5,5,20,19,15,9,12,15,20,15,19,8,17,2
2,18,8,13,6,15,23,20,19,22,14,14,23],
    'x7':
[72,50,74,60,68,58,60,68,61,62,42,62,60,62,65,45,75,64,62,60,63,63,61,5
3,55,68,51,57,57,39,60,60,58,64,59,57,45,70],
    'y1':
[6.8,7.2,6.8,6.8,7.2,7.0,7.0,7.0,6.8,7.0,7.4,7.2,8.0,6.8,7.6,7.0,6.6,6.
6,6.8,7.0,7.0,7.2,6.9,7.5,7.0,7.4,7.9,6.8,7.7,7.4,6.9,7.6,7.0,7.6,6.9,7
.5,6.8,7.2],
    'y2':
[489,464,430,362,453,405,420,466,415,413,404,427,372,496,394,446,420,44
7,398,398,485,400,511,430,487,470,380,460,398,415,470,450,500,410,500,4
00,505,522],
    'y3':
[27,30,32,26,23,29,21,28,24,28,23,25,25,30,24,30,30,25,28,27,30,28,33,2
9,29,28,26,32,27,28,27,28,30,29,33,29,28,28],
    'y4':
[8,5,9,6,11,7,10,2,6,7,6,7,3,10,3,11,12,4,11,4,7,6,12,4,9,7,5,5,2,6,11,
10,12,7,21,2,11,9],
    'y5':
[360,348,386,331,391,389,379,362,386,398,400,407,409,350,399,337,357,44
7,381,387,350,388,298,353,370,360,358,348,383,314,348,326,330,331,348,4
21,355,352]
}

df = pd.DataFrame(data)
X = df[['x1','x2','x3','x4','x5','x6','x7']].values
Y = df[['y1','y2','y3','y4','y5']].values

loo = LeaveOneOut()
max_comp = min(6, X.shape[0]-1)
avg_r2_cv = []
```

```python
for n in range(1, max_comp+1):
    pls = PLSRegression(n_components=n)
    y_pred = cross_val_predict(pls, X, Y, cv=loo)
    r2_list = [r2_score(Y[:, i], y_pred[:, i]) for i in
range(Y.shape[1])]
    avg_r2 = np.mean(r2_list)
    avg_r2_cv.append(avg_r2)
    print(f"n={n}, mean_cv_R2={avg_r2:.4f}, each={np.round(r2_list,4)}")

best_n = np.argmax(avg_r2_cv) + 1
print(f"\n最佳成分数 = {best_n}\n")

scaler_x = StandardScaler()
scaler_y = StandardScaler()
X_std = scaler_x.fit_transform(X)
Y_std = scaler_y.fit_transform(Y)

pls_final = PLSRegression(n_components=best_n)
pls_final.fit(X_std, Y_std)

coef_std = pls_final.coef_
x_mean = scaler_x.mean_
x_scale = scaler_x.scale_
y_mean = scaler_y.mean_
y_scale = scaler_y.scale_

raw_coef = np.zeros_like(coef_std)
for i in range(Y.shape[1]):
    raw_coef[i, :] = y_scale[i] * coef_std[i, :] / x_scale

raw_intercept = y_mean - np.sum(raw_coef * x_mean, axis=1)

Y_pred_std = pls_final.predict(X_std)
Y_pred = scaler_y.inverse_transform(Y_pred_std)
train_r2 = [r2_score(Y[:, i], Y_pred[:, i]) for i in range(Y.shape[1])]

print("=== 最终回归方程 ===\n")
for i, name in enumerate(['y1','y2','y3','y4','y5']):
    eq = f"{name} = {raw_intercept[i]:.4f}"
    for j, var in enumerate(['x1','x2','x3','x4','x5','x6','x7']):
        eq += f" {raw_coef[i,j]:+.4f}*{var}"
    print(eq)
    print(f"    R²_train = {train_r2[i]:.4f}\n")
print(f"平均训练 R² = {np.mean(train_r2):.4f}")
```